Reporte Técnico | fast-grep + Harnesses de Agentes | Gastón Milano / Wany | 6 abril 2026

fast-grep: Búsqueda ultrarrápida como infraestructura para agentes

En una línea: fast-grep es 7-12x más rápido que ripgrep usando un índice de n-grams con frecuencias adaptativas. Pero más importante: establece la infraestructura de búsqueda que cualquier harness de agentes necesita para escalar.

1. Los números — Linux kernel 6.6 (81,690 archivos)

Patrón	grep -r	ag	ripgrep	fg sin índice	fg + índice	Speedup
`EXPORT_SYMBOL`	12.2s	2.4s	1.7s	2.4s	**334ms**	**5.2x**
`printk`	8.3s	1.9s	2.2s	2.4s	**218ms**	**10.2x**
`TODO`	11.0s	1.9s	2.1s	2.3s	**172ms**	**12.0x**
`static.*inline`	11.5s	2.2s	2.3s	2.5s	**410ms**	**5.6x**
`int main`	16.9s	2.6s	2.3s	2.5s	**320ms**	**7.2x**
`container_of`	8.4s	2.9s	3.8s	2.7s	**395ms**	**9.7x**

Promedio: 7.6x sobre ripgrep con índice. Sin índice: empata con ripgrep.

2. Cómo funciona — 5 técnicas

Sparse N-grams con tabla de frecuencias adaptativa

Los sistemas clásicos usan trigramas fijos. fast-grep analiza la frecuencia de pares de caracteres en el corpus específico y genera n-grams de longitud variable basados en rareza. El índice es automáticamente óptimo para el tipo de código que busca.

Position Masks (Algoritmo Blackbird)

Por cada n-gram se almacenan 2 bytes extra: posición módulo 8 y bloom filter del siguiente carácter. Permite descartar candidatos sin leer el archivo. **Resultado: 0.42% tasa de falsos positivos.**

Índice persistente con mmap

```
ngrams.lookup   6.4MB — tabla hash, búsqueda binaria en RAM
ngrams.postings 757MB — posting lists via mmap (no se carga a RAM)
docids.bin      4.3MB — paths de documentos

Load time: 17ms
```

Line-level index con byte offsets

El índice almacena `(doc_id, line_no, byte_offset)` por n-gram. El verify lee solo las líneas candidatas (~80 bytes) en vez de archivos completos (~500KB avg).

Método	I/O en verify	Tiempo
File-level	5.889 archivos × 500KB = 2.9GB	235ms
Line-level	5.889 líneas × 80B = 0.47MB	**~150ms**

Updates incrementales

Operación	Tiempo
Build completo (81,690 files)	~57s
Update incremental (10 archivos)	**707ms**
Speedup vs rebuild	**75x más rápido**

3. Por qué esto importa para harnesses de agentes

> *"The bottleneck has moved. It is no longer writing code. It is everything that comes after: validating behavior, catching regressions, debugging failures." — AgentCore, AWS 2026*

Un agente de código moderno hace ciclos continuos de edición-búsqueda-validación. En 100 ciclos/hora, la búsqueda ocurre cientos de veces.

Escenario	ripgrep	fast-grep	Ahorro
100 búsquedas de contexto	200s (3.3 min)	**25s**	175s
1,000 iteraciones del harness	~33 min	**~4 min**	2.2 horas/día
5 búsquedas de validación/ciclo	10s/ciclo	**1.2s/ciclo**	88%

Búsqueda como tool del agente (NLAH)

```
## Tool: search_codebase
Description: Search codebase with regex, returns files + line numbers
Speed: ~250ms for 80,000 files (12x faster than grep alternatives)
Input: { pattern: string, corpus_path: string }
Output: { matches: [{file, line_no, content}], count: int }
```

Validación continua de invariantes

El paradigma Meta-Harness (Stanford/MIT, 2026) requiere correr búsquedas de regresión en cada iteración:

```
Por cada iteración:
  grep "FIXME|HACK|TODO"       → deuda introducida
  grep "panic!|unwrap()"       → manejo inseguro de errores
  grep "unsafe"                → código peligroso nuevo
  grep "{known_bug_pattern}"   → regresiones conocidas
```

Con fast-grep: 4 búsquedas = ~1s. Con ripgrep: ~8s. La diferencia determina si el harness puede validar continuamente o debe elegir entre velocidad y thoroughness.

Company as Code como KB buscable

En el modelo Company as Code (Wany), toda la empresa vive en un repositorio. Los agentes buscan continuamente en el conocimiento codificado: decisiones de arquitectura, policies, precedentes. fast-grep con índice incremental es la capa de búsqueda natural para una Company KB de 10,000-100,000 archivos.

4. Roadmap

Feature	Estado	Impacto
Sparse n-grams + adaptive frequencies	✅ Producción	Base del sistema
Position masks (Blackbird)	✅ Producción	0.42% false positives
Persistent index + mmap + incremental	✅ Producción	17ms load, 75x update
Line-level index con byte offsets	✅ Nuevo	+40% patrones raros
Adaptive verify strategy	🔄 En desarrollo	+20-30% todos los casos
MCP Server (tool para agentes)	📋 Pendiente	Integración directa en harnesses
Company KB integration	📋 Pendiente	Wany KB como backend

fast-grep no es solo un grep más rápido. Es la infraestructura de búsqueda que permite a los harnesses de agentes operar a la velocidad del pensamiento — buscando contexto, validando invariantes y detectando regresiones sin introducir latencia perceptible en el ciclo de razonamiento del agente.

Repo: github.com/gmilano/fast-grep-rust | MIT | Hardware: M1 Pro 32GB | Corpus: Linux kernel 6.6, 81,690 files